웹 기술 이론 3장

CSS 레이아웃 이론 — 상자·흐름·정렬

박스 모델, 일반 흐름, 포지셔닝, Flexbox와 Grid의 사고 방식

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

학습 목표

① 박스 모델 네 겹과 box-sizing의 차이를 정확히 계산할 수 있다.

② 일반 흐름(normal flow)이 무엇인지 설명하고, 그것을 벗어나는 방법을 안다.

③ 포함 블록(containing block) 개념으로 absolute의 기준을 설명할 수 있다.

④ Flexbox의 주축·교차축 모델로 정렬을 예측할 수 있다.

⑤ Flexbox와 Grid를 상황에 맞게 고를 수 있다.

이론 3장

핵심 키워드

박스 모델 · box-sizing · 일반 흐름 · 마진 상쇄 · 포함 블록 · BFC · flex · grid · fr · minmax

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 3장 · 1

모든 것은 상자다

CSS 레이아웃의 출발점은 모든 요소가 네모난 상자라는 것입니다.

모든 것은 상자다

1. 둥근 버튼도, 원형 아바타도, 실제로는 사각형 상자에 border-radius 를 준 것입니다.
2. 배치를 계산할 때 브라우저가 보는 것은 언제나 사각형입니다.

같은 코드, 다른 결과

```
.box { width: 200px; padding: 20px; border: 1px solid; }

/* content-box (기본값) — width는 "내용"의 폭 */
실제 폭 = 200 + 20 + 20 + 1 + 1 = 242px

/* border-box — width는 "테두리까지"의 폭 */
실제 폭 = 200px
내용 폭 = 200 - 20 - 20 - 1 - 1 = 158px
```

1.1 box-sizing이 바꾸는 것

1. 거의 모든 실무 프로젝트가 CSS 첫 줄에 * { box-sizing: border-box; } 를 씁니다.
2. 3등분한 칸에 padding을 주면 합이 100%를 넘어 넘치는 문제가 이것 하나로 사라지기 때문입니다.
3. max-width 에 패딩을 포함시키지 마세요.
4. border-box에서는 패딩이 바뀌면 내용 폭까지 흔들립니다.
5. 컨테이너는 max-width: calc(내용폭 + 2 * var(--pad)) 처럼 계산해 두어야 화면 폭이 바뀌어도 내용 폭이 일정합니다.

위아래 마진은 겹칩니다

```
.a { margin-bottom: 30px; }
.b { margin-top: 20px; }
/* 사이 간격 = 50px이 아니라 30px (큰 쪽만 남는다) */

/* 부모-자식 사이에서도 일어납니다 */
.parent { margin-top: 0; }
.parent .first-child { margin-top: 40px; }
/* 자식의 마진이 부모 밖으로 새어 나가 부모가 40px 밀린다 */
```

1.2 마진 상쇄

1. 마진 상쇄는 수직 마진에서만 일어나며, 좌우 마진은 겹치지 않습니다.
2. 부모에 padding·border·overflow: hidden·display: flex 중 하나라도 있으면 상쇄가 막힙니다.
3. Flexbox와 Grid 안에서는 마진 상쇄가 아예 일어나지 않습니다 — 이것도 요즘 레이아웃이 편해진 이유 중 하나입니다.

이론 3장 · 2

일반 흐름(normal flow)

아무 배치 속성도 주지 않았을 때 요소가 놓이는 기본 규칙을 일반 흐름이라고 합니다.

일반 흐름(normal flow)

	블록(block)	인라인(inline)
방향	세로로 쌓인다	가로로 이어진다
너비	부모의 가로 전체	내용만큼
width/height	적용됨	적용 안 됨
수직 margin/padding	margin 적용, 다음 요소를 민다	margin 무시, padding은 겹쳐 보임
대표	div p h1 ul li section	span a strong em img

레이아웃 작업이란 사실상 "일반 흐름을 어디까지 유지하고 어디서 벗어날까"를 정하는 일입니다.

inline-block — 둘의 절충

```css
.tag {
  display: inline-block;   /* 옆으로 이어지되 크기를 가질 수 있다 */
  width: 80px;
  padding: 6px 12px;
}

/* inline-block 요소 사이에는 정체불명의 4px 틈이 생깁니다.
   소스의 줄바꿈이 공백 문자로 해석되기 때문입니다.
   → 요즘은 그냥 Flexbox를 씁니다. 이 문제가 없습니다. */
```

이론 3장 · 3

흐름을 벗어나기 — 포지셔닝

흐름을 벗어나기 — 포지셔닝

position	기준(포함 블록)	흐름에 남는가
static	— (위치 지정 안 함)	예
relative	원래 자기 자리	예 (원래 자리를 계속 차지)
absolute	가장 가까운, static이 아닌 조상	아니오 (다른 요소가 자리를 메운다)
fixed	뷰포트(화면)	아니오
sticky	스크롤 컨테이너 + 지정 위치	예

3.1 포함 블록 — absolute의 핵심

1. absolute 요소는 "가장 가까운, position이 static이 아닌 조상" 을 기준으로 위치가 정해집니다.

2. 그런 조상이 하나도 없으면 문서 전체(초기 포함 블록)가 기준이 됩니다.

relative-absolute 짝

```css
.card {
  position: relative;   /* ← 기준점 선언. 이 한 줄이 핵심 */
}
.card .badge {
  position: absolute;
  top: -8px;
  right: -8px;
}

/* .card에 relative가 없으면 배지가 화면 구석으로 날아갑니다.
   "배지가 엉뚱한 데 있어요"의 원인 1위입니다. */
```

3.1 포함 블록 — absolute의 핵심

1. transform·filter·will-change 가 걸린 요소도 fixed 자식의 기준이 됩니다.
2. 그래서 애니메이션을 주었더니 position: fixed 인 모달이 갑자기 이상한 곳에 붙는 일이 생깁니다.
3. 알아 두면 나중에 몇 시간을 아낍니다.

3.2 쌓임 맥락(stacking context)

1. **z-index 는 단순히 숫자가 큰 것이 위로 오는 것이 아닙니다.**

2. **쌓임 맥락이라는 계층이 먼저 있고, 그 안에서만 z-index가 비교됩니다.**

z-index 999가 z-index 1에 지는 경우

```
.parent-a { position: relative; z-index: 1; }
.parent-a .child { position: relative; z-index: 999; }

.parent-b { position: relative; z-index: 2; }

/* .child(999)는 .parent-b(2)보다 아래에 있습니다.
   .child는 .parent-a라는 상자 안에 갇혀 있고,
   그 상자 자체가 z-index 1이기 때문입니다. */
```

3.2 쌓임 맥락(stacking context)

1. 쌓임 맥락은 position + z-index, opacity < 1, transform, filter, will-change 등이 만듭니다.

2. "z-index를 아무리 올려도 안 올라온다"면 부모가 만든 쌓임 맥락에 갇힌 것입니다.

이론 3장 · 4

Flexbox — 한 방향 정렬 모델

Flexbox의 사고 모델은 단순합니다.

Flexbox — 한 방향 정렬 모델

flex-direction	주축	교차축
row (기본)	가로 →	세로 ↓
row-reverse	가로 ←	세로 ↓
column	세로 ↓	가로 →
column-reverse	세로 ↑	가로 →

주축(main axis) 하나를 정하고, 그 방향으로 요소를 늘어놓은 뒤, 주축과 교차축 각각에서 정렬합니다.

두 속성만 정확히 기억하면 됩니다

정리

1. **justify-content**
   = 주축 정렬 (요소가 늘어서는 방향)

2. **align-items**
   = 교차축 정렬 (직각 방향)

3. **flex-direction: column 이면 둘의 방향이 뒤바뀝니다. "안 먹는 것 같다" 싶으면 direction부터 확인하세요.**

flex: grow shrink basis

```
flex: 1;            /* = 1 1 0    남는 공간을 균등하게 나눠 가진다 */
flex: 1 1 220px;    /* 기본 220px, 늘어나도 줄어도 된다 — 반응형 카드 */
flex: 0 0 200px;    /* 무슨 일이 있어도 200px 고정 — 사이드바 */
flex: auto;         /* = 1 1 auto  내용 크기를 기준으로 늘어난다 */
flex: none;         /* = 0 0 auto  내용 크기 그대로 고정 */
```

4.1 flex 축약형 읽기 (1/2)

1. flex-wrap 의 기본값은 nowrap 입니다.
2. 그래서 좁은 화면에서 카드가 찌그러집니다.
3. flex-wrap: wrap + flex: 1 1 220px 조합이면 미디어쿼리 없이도 반응형이 됩니다.

4.1 flex 축약형 읽기 (2/2)

1. **flex 항목의 min-width 기본값은 auto 라서 내용보다 작아지지 않습니다.**
2. **긴 텍스트나 넓은 표가 든 항목이 안 줄어들면 min-width: 0 을 주세요.**
3. **Grid의 minmax(0, 1fr) 과 같은 이유의 문제입니다.**

이론 3장 · 5

Grid — 2차원 격자 모델

Grid는 틀을 먼저 정하고 그 안에 내용을 채우는 방식입니다.

트랙 정의

```css
grid-template-columns: 200px 1fr 200px;    /* 고정 · 나머지 · 고정 */
grid-template-columns: repeat(3, 1fr);     /* 균등 3등분 */
grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  /* 그 자체로 반응형 */

grid-template-rows: auto 1fr auto;         /* 머리 · 본문(남는 만큼) · 바닥 */
gap: 16px 24px;                            /* 행 간격 · 열 간격 */
```

> **Flexbox가 "내용이 크기를 정한다"면 Grid는 "틀이 크기를 정한다"에 가깝습니다.**

5.1 fr 과 minmax(0, 1fr)

1. fr 은 남는 공간을 나누는 비율 단위입니다.

2. 그런데 1fr 의 최소 크기는 auto(=내용 크기)라서, 안에 든 것이 크면 칸이 줄어들지 않습니다.

실무에서 반드시 쓰는 형태

```
/* ✗ 긴 URL이나 넓은 표가 들어가면 격자가 화면 밖으로 삐져나간다 */
grid-template-columns: repeat(3, 1fr);

/* ✓ 최소를 0으로 못 박아 내용이 커도 칸이 줄어들게 한다 */
grid-template-columns: repeat(3, minmax(0, 1fr));
```

5.2 auto-fit과 auto-fill

	동작	항목이 적을 때
auto-fit	들어가는 만큼 칸을 만들되 빈 칸을 접는다	남은 공간을 항목이 나눠 가져 넓어짐
auto-fill	들어가는 만큼 칸을 만들고 빈 칸을 남긴다	항목이 원래 크기를 유지하고 오른쪽이 빈다

5.2 auto-fit과 auto-fill

1. 대부분 auto-fit 이 원하는 동작입니다.

2. 그리고 이 조합에는 미디어쿼리를 덧붙이지 마세요 — 이미 반응형이라 오히려 동작을 망칩니다.

grid-template-areas — 페이지 뼈대에 최적 (1/2)

```
.layout {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-areas:
    "header header"
    "side   main"
    "footer footer";
  gap: 16px;
}
.layout > header { grid-area: header; }
.layout > aside  { grid-area: side; }
```

grid-template-areas — 페이지 뼈대에 최적 (2/2)

```
.layout > main   { grid-area: main; }
.layout > footer { grid-area: footer; }

/* 반응형: 영역 배치만 다시 그리면 된다 */
@media (max-width: 768px) {
  .layout {
    grid-template-columns: 1fr;
    grid-template-areas: "header" "main" "side" "footer";
  }
}
```

5.3 영역 이름으로 배치하기

1. 이 방식의 장점은 CSS만 보고도 화면 배치가 그려진다는 것입니다.
2. 반응형에서 순서까지 바꿀 수 있어 사이드바를 모바일에서 본문 아래로 내리는 것도 한 줄입니다.
3. 다만 grid-template-areas 로 시각적 순서를 바꾸면 키보드 이동 순서(코드 순서)와 어긋납니다.
4. 화면에서는 본문이 먼저인데 Tab을 누르면 사이드바로 가는 식입니다.
5. 순서를 크게 바꿀 때는 HTML 순서 자체를 고치는 편이 낫습니다.

이론 3장 · 6

무엇을 언제 쓸 것인가

무엇을 언제 쓸 것인가

상황	도구	이유
내비게이션 메뉴	Flexbox	한 줄, 내용 크기가 제각각
버튼 묶음	Flexbox	한 방향 + gap
가운데 정렬	Flexbox	세 줄이면 끝
카드 나열 (한 줄 기준)	Flexbox + wrap	내용에 따라 유연하게
사진 갤러리	Grid + auto-fit	균등한 격자
페이지 전체 뼈대	Grid + areas	행과 열이 모두 중요
대시보드 (칸 합치기)	Grid + span	2차원 배치

실무의 기본형

정리

1. **큰 틀은 Grid, 각 칸 안은 Flexbox.**
   이 조합이 가장 흔하고 가장 잘 동작합니다.
2. **float 는 이제 쓰지 않습니다.**
   글 안에 이미지를 흘려 넣는 원래 용도에만 남았습니다.
3. **표로 화면을 배치하지 않습니다.**
   화면낭독기가 표로 읽고, 반응형이 불가능합니다.

웹 기술 이론 3장 마무리

3장 마무리

CSS 레이아웃 이론 — 상자·흐름·정렬 — 본문과 도해는 학습사이트에서 다시 읽습니다

- ✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시
- ✓ 함께 볼 주차: 4주차 · 5주차 · 6주차
- ✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다